

ME

LOL, THIS IS A TITLE

Contents

<i>Introduction</i>	5
---------------------	---

Introduction

As an initial proof of concept work was done with raw MC DAOD files. The goal was to see if a model could be trained to identify Z-decays better than the single dimensional selection used by the group.

This was done by adapting existing python code originally written by a former student of the group.[cite Ceren?]

The algorithm was very simple. looking in the root file we looped over every single event, events that would contain a number of leptons, then for every pair of leptons we would save the needed information about these. For pre-processing for the isolation model the algorithm would save every single lepton individually.

This early code was slower than established ATLAS frameworks, although great speedup was achieved using the multiprocessing library for python.

The central drawback of this was that it was very difficult to get information about the events with regard to the established model. Meaning that it would be difficult to provide any information about which of the cuts made in the established model were too strict.

ATLAS Data

The model

In this section I will describe the model, the data pre-processing and the training of the model.

The goal of the model is to boil a large parameter space down to a single number; the probability that this lepton pair comes from a Z-boson. A single number has a lot of benefits over the existing, cuts-based method. First and foremost it is a lot easier to loosen the selection, this is useful if one wants more statistics. especially in the case of the $H \rightarrow Z\gamma$ decay where we know that the background from $Z \rightarrow \ell\ell$ in the final Higgs signal is minimal. [SOURCE SOURCE SOURCE].

overall structure

It is important that the model does not train on the invariant mass of the leptons. This is because for Z-selection, we expect a peak at the Z-mass. So should the model simply learn this mass, then it would be useless for the selection.

To combat this we are very selective with what variables we feed into the model and when we use them.

We essentially train two models, (three for the case of $Z \rightarrow \mu\mu$). One model is trained on the isolation of the leptons, the other is trained on the identification of the leptons. Then we feed the values from these two models into a third model, along with information on how close the origins of the lepton tracks are from each other.

PID - Particle Identification For electrons we do not need to train a model for the PID, as there already exists a model for electron identification that performs very well.

<http://cds.cern.ch/record/2809283/files/ATL-PHYS-PUB-2022-022.pdf>

This model will not only provide an electron-score, but also other scores that attempts to give information on what type of electron it is.

meaning that the meta-model will not only be provided with the identification score that the leptons are in fact electrons. But might also know that the lepton comes from a number of background events that are outside of the area we are interested in.

Data pre-processing

First we need to pre-process the data. Data and MC from the ATLAS detector comes in the form of a ROOT file. The cuts-based selection is then performed directly on these files, returning an Ntuple with the specific events that might be needed for future data processing and only containing the variables that are needed.

Our model will need additional variables and it will need more events, if nothing else for background for the training. So first step is to adapt the C++ code that does the cuts-based selection to output an Ntuple with the variables that we need.

The adaptation that we made does the following:

1. Load the ROOT file.
2. Apply the cuts-based selection.
3. for events that pass the selection, save the events with additional variables.
4. for events that do not pass the selection, run the algorithm again, this time ignoring the single cut that failed. Then save the event based on a condition that we define.

The adaptation will not only provide the benefit of having more data, but also having the information of what cut failed for the event. This means that even without directly implementing the model, we might be able to provide feedback on where the cuts-based selection might be too strict.

For signal MC we use $Z \rightarrow \ell\ell$ and for background we use $t\bar{t}$. We feed this through the adapted code and get two Ntuples.

background

For isolation background is simple. we use the $Z \rightarrow \ell\ell$ and $t\bar{t}$ MC Ntuples that all contain pairs of leptons. We then unpair this data to look at individual leptons that can be tagged as "Isolated" or not. the isolated leptons are considered signal and everything else is background.

for the Z-model in the $Z \rightarrow ee$ case we use the same samples. but this time to specify signal we first need to know that the pair is from a Z-boson. This is needed because with the loosed selection we only see about 70% true-Z pairst from the $Z \rightarrow ee$ sample. background is everything from the $t\bar{t}$ sample.

Feasablility of opening cuts

charge this is ugly, fix it

For the version of the framework where we first, run the framework as normal and only if they fail run it again; removing cuts untill we have an event to save.

In Ttbar data we see that the ratio of events saved with same charge to the total number of events saved is 0.17450071447483853.

And for Zee data we see that the ratio of false sign measurements are: 0.010364901892708698 for ℓ_1 and 0.06765421692365889 for ℓ_2 .

yielding that only 0.35265877647357798% of $Z \rightarrow ee$ events saved with this modified framework are false sign events.

The conclusion is that opening charge, while an easy cut to open, is not a cut where there is a lot of potential gain.